Can you explain how a decoder only architecture functions?

So if you are asked this question, especially in generative AI related interviews or in NLP related

interviews, you can potentially start talking about Transformers in case you have a good hold on transformer

architectures, start from there.

Show more

Your explanation captures the high-level intuition (encoder = understanding, decoder = generation), but it contains a few important technical inaccuracies that could hurt in a Generative AI or NLP interview. Here's a more accurate interview-ready explanation.

How a Decoder-Only Architecture Works (Interview Answer)

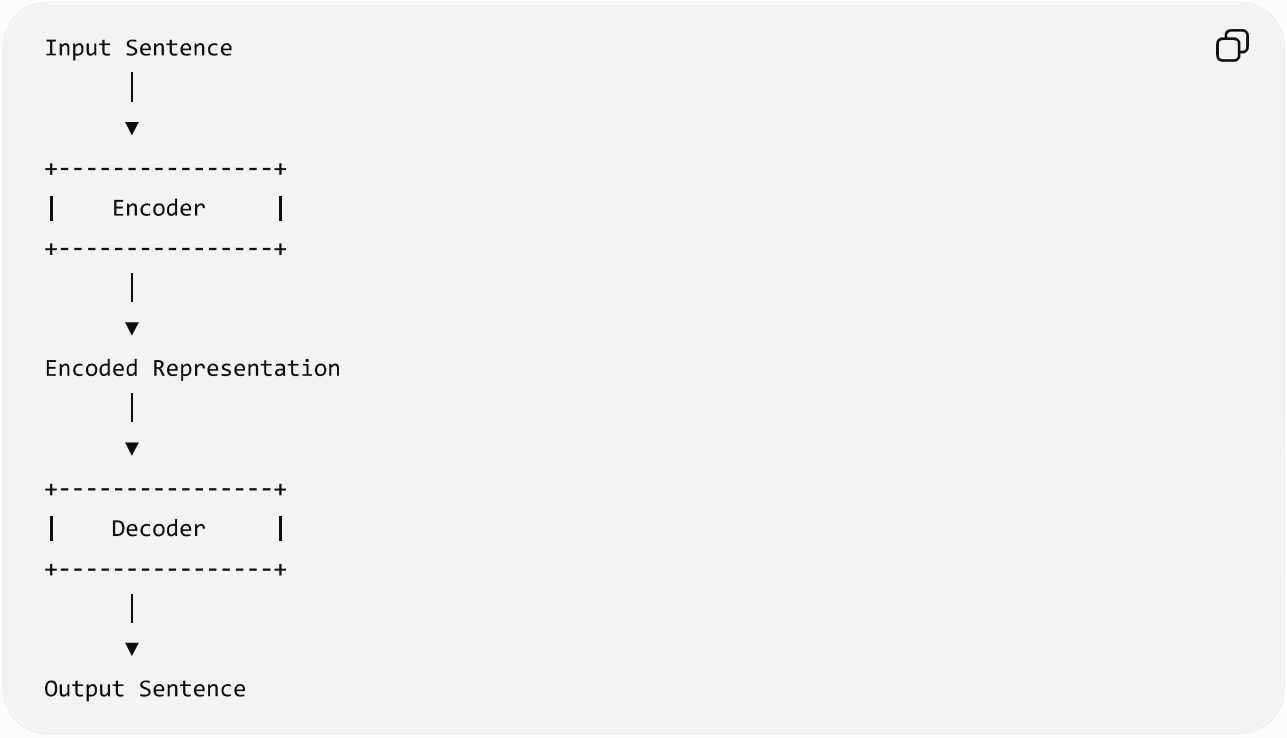
A **decoder-only architecture** is a Transformer model designed primarily for **autoregressive text generation**, where it predicts **the next token based only on the previous tokens**.

Examples include:

- GPT-1
- GPT-2
- GPT-3
- GPT-4 (widely believed to be decoder-based with additional improvements, though OpenAI has not publicly disclosed the full architecture)
- Llama
- Mistral
- Qwen
- DeepSeek
- Claude (architecture details are not fully public)

Start from Transformers

A Transformer consists of two major components:



Originally introduced in the paper:

Attention Is All You Need (2017)

—not 2016 or 2007.

Encoder vs Decoder

Encoder	Decoder
Understands input	Generates output
Bidirectional attention	Causal (masked) attention
Used for classification	Used for generation
Reads entire sentence	Generates one token at a time

Example:

Input:

```
How are you?
```

Encoder creates contextual embeddings.

Decoder generates:

```
आप कैसे हो?
```

What changed?

Researchers discovered that different tasks require different architectures.

Instead of always using both encoder and decoder, they built:

```
Encoder Only
```

Examples

- BERT
- RoBERTa
- ALBERT
- DistilBERT

Used for

- Classification
- Named Entity Recognition
- Sentiment Analysis
- Search
- Embeddings

```
Decoder Only
```

Examples

- GPT
- Llama
- Mistral
- Qwen

Used for

- Chatbots
- Content Generation
- Code Generation
- Story Writing
- Translation
- Summarization

But where does the input go?

This is where many people get confused.

Since there is no encoder,

the prompt itself is directly fed into the decoder.

```
Prompt

"What is Machine Learning?"

        |
        ▼

Tokenizer

        |

Token IDs

        |

Embedding Layer

        |

Positional Encoding

        |

Decoder Block 1

        |

Decoder Block 2

        |
```

```
...
        |
Next Token Prediction
```

The decoder never receives an encoder output.

It simply receives the prompt.

How does the decoder generate text?

Suppose the prompt is

```
The capital of France is
```

The model predicts

```
Paris
```

Now the sentence becomes

```
The capital of France is Paris
```

Next prediction is

```
.
```

Then

```
The capital of France is Paris.
```

Next

```
It
```

and so on.

Generation happens **one token at a time**.

This process is called **autoregressive generation**.

Why can it not see future words?

Every decoder block uses

Masked Self-Attention

```
I love playing football

Predicting "playing"

Can attend to:

I
love

Cannot attend to

football
```

Future tokens are masked.

This prevents cheating during training.

Hence it learns next-token prediction.

Internal Decoder Flow

```
Prompt

↓

Tokenizer

↓

Embedding

↓

Positional Encoding

↓

Masked Multi-Head Attention

↓
```

```
Feed Forward Network

↓

Masked Multi-Head Attention

↓

Feed Forward Network

↓

...

↓

Linear Layer

↓

Softmax

↓

Probability Distribution

↓

Next Token
```

This repeats until

- End-of-sequence token
- Maximum length
- User stops generation

What happens during training?

During pretraining,

Input:

```
The cat sat on the
```

Target:

```
mat
```

The model learns

```
P(mat | The cat sat on the)
```

Millions or billions of such examples teach the model language patterns.

Is there a vector database inside GPT?

This is one of the biggest misconceptions.

No.

A pretrained GPT model **does not search a vector database internally** to answer questions.

Instead:

```
Prompt

↓

Tokenizer

↓

Transformer Layers

↓

Neural Network Weights

↓

Next Token Prediction
```

All knowledge learned during pretraining is stored in the **model's parameters (weights)**, not in a vector database.

When is a vector database used?

Vector databases come into play in **Retrieval-Augmented Generation (RAG)**, which is a system built around the language model.

Example:

```
User Question

↓

Embedding Model

↓

Vector Database
(Qdrant, Pinecone, FAISS, Milvus)

↓

Relevant Documents Retrieved

↓

Prompt Augmentation

↓

GPT / Llama / Mistral

↓

Answer
```

Here, the vector database retrieves external documents, and those documents are added to the prompt before the decoder generates an answer.

This retrieval step is **outside** the decoder-only architecture itself.

Interview Summary (2-minute answer)

> A decoder-only architecture is a Transformer designed for autoregressive text generation. Unlike the original encoder-decoder Transformer, it removes the encoder and feeds the user's prompt directly into the decoder. The decoder uses masked self-attention so that each token attends only to previous tokens, preventing access to future tokens. It predicts one token at a time, appends that token to the context, and repeats until the response is complete. Models such as GPT, Llama, Mistral, and Qwen

> use this architecture. It's important to note that these models do not internally search a vector database; their knowledge is encoded in their learned weights. Vector databases are used only in external retrieval systems like RAG, where retrieved documents are added to the prompt before generation.

Corrections to the original explanation

- ✅ Transformers were introduced in **2017**, not 2016 or 2007.
- ✅ GPT models do **not** search an internal vector database during inference.
- ✅ FAISS, Qdrant, Pinecone, etc., are components of **RAG systems**, not part of the decoder architecture.
- ✅ The prompt is tokenized, embedded, and passed directly into the decoder.
- ✅ Decoder-only models generate text through **autoregressive next-token prediction** using **masked self-attention**.